

KanbanApp	
Arquitetura do Sistema - Architecture Notebook	Data: 13/06/2025

KanbanApp

Architecture Notebook

1. Propósito

Este documento descreve a arquitetura do KanbanApp, apresentando os objetivos arquiteturais, requisitos relevantes, decisões de projeto, mecanismos arquiteturais, abstrações principais e diferentes visões da arquitetura adotada para o desenvolvimento do sistema.

2. Objetivos e Filosofia da Arquitetura

A filosofia principal da arquitetura do KanbanApp é manter o código simples, organizado e fácil de expandir. Hoje o sistema funciona como um aplicativo desktop individual que usa o banco de dados local, mas isso serve apenas para a fase de protótipo e validação. A estrutura foi pensada desde o começo para não ficar presa a um cenário com apenas um usuário. O sistema foi dividido em camadas lógicas que permitem no futuro colocar a aplicação em rede, conectar várias pessoas ao mesmo tempo e trocar o banco de dados local por uma solução corporativa maior sem precisar reescrever o código inteiro.

Para garantir a construção do sistema, a arquitetura segue quatro diretrizes. A primeira é a Separação Estrita de Responsabilidades (SoC), onde o sistema isola de forma categórica a Interface Gráfica (View), a Intermediação de Fluxo (Controller), as Regras de Negócio (Service) e a Camada de Persistência (Repository). Esse isolamento garante que modificações na interface CustomTkinter não gerem efeitos colaterais nas rotinas analíticas, como o cálculo automatizado das métricas de Cycle Time e Lead Time ou a validação do limite Work In Progress, além de viabilizar a cobertura abrangente por testes unitários independentes. A segunda é a Modularidade Voltada à Escalabilidade de Dados, tratando a camada de acesso a dados como um componente intercambiável. Embora o protótipo utilize o módulo nativo sqlite3 para garantir portabilidade imediata e uma infraestrutura simples e direta, o encapsulamento das consultas dentro de classes de repositório isoladas assegura que a substituição do motor de banco de dados ocorra com impacto nulo sobre as regras de negócio. O terceiro ponto é a Transparência e Controle no Acesso ao Modelo Relacional. A persistência e o controle transacional são operados por meio de instruções SQL nativas e estruturadas diretamente na camada apropriada. A decisão de não utilizar frameworks complexos de mapeamento objeto relacional nesta fase assegura a máxima eficiência no processamento de dados locais e a fidelidade estrita ao esquema físico projetado. Por fim, o Gerenciamento Orientado a Eventos Intermediados define que a interface com o usuário não possui comunicação direta com os serviços de processamento de dados. A camada Controller atua como o único ponto de interceptação dos estímulos da interface, validando os parâmetros de entrada antes de acionar o núcleo de negócios, o que estabelece uma fundação segura para a futura implementação de concorrência e controle de acesso simultâneo.

3. Requisitos Arquiteturalmente Significativos

Os requisitos arquiteturalmente significativos que fundamentam as decisões e restrições deste documento encontram-se especificados no artefato número 3 System-Wide Requirements Specification. A arquitetura aqui definida foi estruturada para suportar diretamente a implementação das funcionalidades centrais descritas nesses artefatos, como a criação e gerenciamento de quadros Kanban, o controle automatizado dos limites de Work In Progress nas colunas e o cálculo e extração das métricas operacionais de Cycle Time e Lead Time. Para a validação completa das regras de negócio, fluxos de exceção e restrições de interface gráfica que originaram o isolamento em camadas.

4. Decisões, Restrições e Justificativas

Uso do SQLite com SQL Nativo (Sem ORM)

Decisão e Restrição: O sistema utiliza o banco de dados SQLite nativo do Python e o acesso aos dados deve ser feito apenas com consultas SQL puras, proibindo o uso de ferramentas complexas de mapeamento objeto relacional.

KanbanApp	
Arquitetura do Sistema - Architecture Notebook	Data: 13/06/2025

Justificativa: Isso faz o aplicativo funcionar de forma autônoma na mesma hora, sem obrigar o usuário a instalar ou configurar um servidor por fora. O SQL puro também deixa o código mais leve, mais rápido e dá o controle total sobre o modelo físico das tabelas que projetamos.

Isolamento Completo da Interface Gráfica

Decisão e Restrição: É proibido instanciar conexões com o banco de dados ou rodar regras de negócio diretamente dentro dos arquivos visuais construídos com o CustomTkinter. A comunicação obrigatoriamente passa pela camada do Controller.

Justificativa: Isso "blinda" o núcleo do sistema. Se precisarmos redesenhar uma tela inteira no futuro, o cálculo das métricas de Cycle Time e a validação do Work In Progress vão continuar intactos. A interface serve apenas para exibir informações

Uso Obrigatório de Caminhos Relativos

Decisão e Restrição: Fica proibida a utilização de caminhos absolutos no código fonte para referenciar o arquivo do banco de dados ou arquivos estáticos da interface.

Justificativa: Como o sistema será empacotado pelo PyInstaller para rodar em diferentes sistemas e em pastas escolhidas livremente pelo usuário, o uso de caminhos relativos garante que o executável encontre tudo o que precisa na sua própria pasta base, garantindo a portabilidade do software.

Diretrizes práticas para manter a arquitetura organizada durante a construção do código, os desenvolvedores devem seguir estas regras:

- **O que fazer:** Coloque as validações lógicas, como as checagens de limite das colunas, exclusivamente dentro da camada de serviço.
- **O que não fazer:** Não crie ou altere colunas diretamente no arquivo do banco de dados sem antes revisar a modelagem oficial do projeto
- **O que fazer:** Mantenha todo o código que conversa com o banco de dados encapsulado nas classes da camada de repositório.
- **O que não fazer:** Não coloque regras de negócio escondidas nos formulários ou botões da tela.
- **O que fazer:** Garanta o fechamento seguro da conexão com o SQLite ao final de cada execução no repositório.
- **O que não fazer:** Não faça a camada de interface falar direto com a camada de repositório, sempre passe pelo controlador..

5. Mecanismos Arquiteturais

Abaixo estão listados os mecanismos principais da aplicação e como eles funcionam no cenário atual do projeto.

Persistência Local de Dados

Este mecanismo lida com o armazenamento seguro de todas as informações do sistema, como os dados de projetos, limites de colunas e os registros de tempo dos cartões. O estado atual da implementação utiliza o módulo sqlite3 nativo do Python. A principal função desse mecanismo é garantir que as operações de gravação e leitura no banco de dados aconteçam de forma rápida diretamente na máquina do usuário. Ele opera recebendo comandos SQL puros da camada de repositório e garante o fechamento imediato das conexões logo após cada execução, evitando corrupção do arquivo físico e garantindo a integridade dos dados locais.

Isolamento Estrutural em Camadas

O propósito deste mecanismo é organizar o código da aplicação de maneira limpa, separando a interface visual das lógicas de validação e do acesso aos dados. O estado atual é uma estrutura de diretórios rigidamente dividida entre Views, Controllers, Services e Repositories. Sua função é garantir o fluxo unidirecional das informações. A tela construída com CustomTkinter tem a única função de exibir botões e textos. Quando o usuário interage, a interface repassa o comando para o controlador correspondente, que aciona o serviço para validar as

KanbanApp	
Arquitetura do Sistema - Architecture Notebook	Data: 13/06/2025

regras do método Kanban e, por fim, chama o repositório para salvar a mudança.

Tratamento de Eventos da Interface Visual

Este mecanismo gerencia como as ações físicas do usuário chegam até o núcleo lógico do sistema. Atualmente, ele funciona aproveitando o sistema nativo de eventos e *callbacks* (comandos) do próprio framework CustomTkinter, conectando essas interações diretamente aos Controladores da nossa arquitetura. A função dele é capturar um evento, como o clique no botão de mover um cartão para a coluna Fazendo, e transformar isso em um comando lógico estruturado. Ele impede que a tela trave durante o processamento e garante que as validações de Work In Progress ocorram no momento exato da interação.

Empacotamento de Distribuição

O propósito deste mecanismo é transformar o código fonte escrito pela equipe em um programa executável comum que possa rodar em qualquer computador sem exigir configurações técnicas do usuário final. O estado atual utiliza a ferramenta PyInstaller para construir a versão final da aplicação. A função principal desse mecanismo é empacotar o interpretador Python, as bibliotecas gráficas e os mapeamentos de caminhos relativos do banco de dados em um formato autossuficiente, entregando portabilidade .

6. Abstrações Principais

As abstrações principais do KanbanApp traduzem os conceitos do mundo real e os padrões de projeto em estruturas lógicas dentro do código.

A grande abstração é o próprio Modelo de Domínio do Kanban, representado pelas classes de Projeto, Quadro, Coluna e Cartão. Essa abstração transforma um fluxo de trabalho em entidades rastreáveis. O Cartão atua como a unidade fundamental de transporte de dados, carregando as informações e os carimbos de tempo essenciais para gerar as métricas ágeis. A Coluna abstrai as etapas sequenciais do processo produtivo e retém as restrições de capacidade operacional.

No quesito estrutural, a abstração de Repositório esconde toda a complexidade da linguagem SQL e as particularidades do banco de dados do restante da aplicação. Na perspectiva do sistema, salvar uma tarefa ou criar um quadro é uma chamada de método limpa e direta. Isso protege a lógica da aplicação e centraliza a manipulação estrutural dos dados em um único lugar.

A abstração de Serviço encapsula a metodologia Kanban. A interface gráfica e os repositórios não “sabem” como calcular o Cycle Time, o Lead Time ou como conferir se uma coluna estourou seu limite de WIP. A camada de Serviço abstrai todas essas regras matemáticas e lógicas solicitadas pelo usuário e verifica caso a solicitação é válida ou não com base na teoria Kanban.

Por fim, a abstração do Controlador atua como o tradutor de intenções do sistema. Ele abstrai os eventos físicos da interface gráfica gerados pelo CustomTkinter , como o clique de um mouse ou o preenchimento de um formulário. O Controlador transforma essas interações visuais em comandos lógicos que os Serviços e Repositórios irão processar, garantindo que a tela nunca precise conhecer os detalhes internos do funcionamento do banco ou das regras de negócio.

7. Camadas ou Framework Arquitetural

Adota-se um modelo arquitetural híbrido que combina os princípios do padrão Model View Controller com a tradicional Arquitetura em Camadas. Essa abordagem foi escolhida para garantir que a aplicação seja consistente e uniforme em todas as suas funcionalidades. A regra fundamental desse padrão é o fluxo de comunicação estritamente unidirecional, as informações transitam de forma linear e previsível. A interface visual da aplicação não poderá contornar as regras do sistema para acessar os dados armazenados de forma direta, e o banco de dados nunca envia informações para a tela sem antes passar por um tratamento lógico. A integração destes componentes começa pela camada View (Framework CustomTkinter , visual), repassada para a camada Controller (Intermediário principal) e então, chega na camada Service (Regras do sistema). Para concluir o ciclo de execução, a camada de serviços se comunica com a camada Repository (Banco de dados), esses modelos funcionam como transporte de dados entre a View, o Controller, o Service e o Repository, conectando os componentes, facilitando qualquer processo de manutenção e expansão do código fonte.

KanbanApp	
Arquitetura do Sistema - Architecture Notebook	Data: 13/06/2025

8. Visões Arquiteturais

A partir de diferentes perfis técnicos, a arquitetura está documentada a partir de três perspectivas principais.

Visões

Lógica : Na visão lógica do KanbanApp, a estrutura central está contida no diretório principal do código fonte e subdividida em pacotes com responsabilidades definidas. O pacote de modelos abriga as classes que representam entidades como Projetos, Quadros, Colunas e Cartões. A interface visual, concentrada no pacote de views e construída com componentes do CustomTkinter , apenas exibe os dados e emite sinais de interação. Esses sinais são interceptados pelo pacote de controladores, que traduz o clique do usuário em um comando de sistema e o repassa para o pacote de serviços. É no pacote de serviços que residem as regras do sistema, como o cálculo de métricas e o bloqueio de colunas lotadas, por exemplo. Após a aprovação lógica, a execução desce para o pacote de repositórios, que é o único autorizado a montar e executar comandos SQL no banco de dados. A persistência de todas essas estruturas lógicas é materializada pelo arquivo físico do banco de dados, gerado a partir do script de criação formalizado dentro da modelagem relacional.

Operacional: A visão operacional reflete a topologia de implantação contida do protótipo. O sistema inteiro é executado em um único nó físico, que corresponde ao computador pessoal do usuário. Por conta do empacotamento realizado pelo PyInstaller, a aplicação roda sob um único processo principal no sistema operacional. A thread principal da aplicação gerencia o loop de eventos da interface gráfica e executa de forma direta e síncrona as operações de leitura e escrita no arquivo local do banco de dados. Como todas as dependências estão embarcadas no executável gerado, a arquitetura garante a ausência completa de componentes distribuídos, não havendo necessidade de servidores de aplicação externos, APIs na nuvem ou contêineres de banco de dados rodando em segundo plano.

Caso de Uso: A arquitetura evidencia como as funcionalidades ditam o comportamento das camadas internas. O caso de uso de movimentar um cartão de atividade entre colunas é o maior justificador da arquitetura. Quando o usuário tenta essa ação, a interface gráfica aciona o controlador, que por sua vez invoca o serviço correspondente. O serviço consulta o repositório para verificar a capacidade atual da coluna de destino e, se o limite de tarefas não tiver sido atingido, autoriza a movimentação e salva automaticamente o carimbo de tempo que será usado futuramente para o cálculo do Cycle Time.

Outro caso de uso estrutural é a exclusão de um projeto inteiro pelo gerente. Essa ação reflete o alinhamento da interface com a modelagem do banco de dados, acionando as restrições de exclusão em cascata configuradas para remover de forma limpa todos os quadros, colunas e cartões vinculados sem deixar dados sem referência no sistema.

9. Impacto das Ferramentas na Arquitetura

KanbanApp	
Arquitetura do Sistema - Architecture Notebook	Data: 13/06/2025

Ferramenta / Framework	Impacto Arquitetural	Descrição e Efeitos no Sistema
CustomTkinter <i>(Interface Gráfica)</i>	Isolamento da Thread Principal (Separação View-Controller)	O funcionamento baseado no loop de eventos principal (mainloop) e em <i>callbacks</i> forçou a separação rigorosa entre a UI e as regras de negócio. Qualquer interação repassa o comando ao Controller para evitar o congelamento da tela principal durante cálculos de métricas ou consultas ao banco de dados.
sqlite3 Nativo <i>(Acesso a Dados sem ORM)</i>	Mapeamento Manual de Estruturas na Camada de Repositório	A necessidade de executar instruções SQL em formato de texto bruto exigiu a criação de rotinas manuais de conversão estrutural. Os repositórios recebem as linhas do SQLite e as transformam em instâncias das classes de Modelo, mantendo a camada de persistência leve e atrelada ao projeto físico.
PyInstaller <i>(Empacotamento)</i>	Resolução Dinâmica de Caminhos Relativos	O encapsulamento do sistema em um programa executável autônomo inviabilizou o uso de caminhos absolutos no código fonte. A arquitetura precisou embutir mecanismos dinâmicos para localizar o arquivo do banco de dados e as imagens da interface gráfica, independentemente do diretório de instalação.